

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

«Резервирование БД и восстановление по контрольным точкам»

Выполнили:

Матвеев Илья Александрович, студент группы N3351

(подпись)

Проверил:

Карманова Наталия Андреевна, ассистент ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Ход Работы	4
1.1 Задание	4
1.2 Создание резервной копии БД	4
1.3 Внесение случайных изменений и откат к последней контрольной точке	7
1.4 Анализ внесенных изменений	7
1.4.1 Первый способ	7
1.4.2 Второй способ	8
Заключение	9

ВВЕДЕНИЕ

Цель работы – получение навыков по резервированию и восстановлению БД.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить литературу выбранной СУБД по созданию контрольных точек в БД;
- Тестирование созданных контрольных точек;
- Проанализировать возможность анализа и просмотра изменений, в том числе, при помощи системы логирования.

1 ХОД РАБОТЫ

В данном разделе представлено выполнение всех задач, поставленных в данной лабораторной работе, а также тестирование и анализ полученных результатов.

1.1 Задание

Пункты задания звучат следующим образом:

1. Создание резервной копии БД согласно выбранному расписанию.
2. Внести случайные изменения в таблицы созданной вами базы данных (изменения вносятся до момента создания контрольной точки).
3. Продемонстрировать процесс отката к последней контрольной точке. Откатите изменения, выполненные в пункте 2.
4. Проанализируйте возможность анализа/просмотра изменений, которые были «откачены», с помощью системы логирования СУБД или с помощью средств системы резервирования.

1.2 Создание резервной копии БД

Для создания резервной копии БД воспользуемся встроенными инструментами СУБД pgAdmin4. Для того, чтобы резервная копия создавалась по выбранному расписанию (допустим, 1 раз в неделю), воспользуемся встроенным инструментом Windows – планировщиком заданий.

Для создания резервной копии БД выполним следующие действия:

1. Из визуального интерфейса СУБД выберем необходимую базу данных (в моем случае onlineSchool), выберем опцию Backup;
2. В качестве названия файла для бэкапа выберем пользовательскую директорию, а в ней файл восстановления .backup;
3. Создадим резервную копию по нажатию клавиши Save.

После выполнения данных действий и подтверждения выполнения процесса получим готовую версию бэкап-файла для текущей базы данных.

Далее, необходимо создать резервирование по расписанию. В СУБД pgAdmin4 напрямую это сделать нельзя, поэтому воспользуемся планировщиком задач Windows. Для этого выполним следующие действия:

1. Откроем «Планировщик задач», выберем опцию «Создать задачу»;
2. В качестве параметров укажем: «Имя»: «Восстановление_БД»;

3. Сама задача будет запускать bat-файл backup.bat, в котором содержится следующий текст:

```
@echo off
set PGPASSWORD=ПАРОЛЬ
set DATESTAMP=%date:~6,4%-~3,2%-~0,2%
"C:\Program Files\PostgreSQL\17\bin\pg_dump.exe" -U postgres -F c -f
"C:\backups\onlineSchool_%DATESTAMP%.backup" onlineSchool
```

Этот bat-файл запускает восстановление, а каждую новую резервную копию сохраняет, включая в название дату, когда была создана эта копия.

4. Далее, необходимо задать расписание. Для этого в той же задаче создадим триггер, который еженедельно будет выполнять данную задачу. Параметры для триггера представлены на рисунке 1.

Создание триггера

Начать задачу: По расписанию

Настройки

☐ Однократно

☐ Ежедневно

☒ Еженедельно

☐ Ежемесячно

Начать: 23.10.2025 15:50:00 ☐ Синхр. по поясам

Повторять каждую 1 неделю в:

☐ воскресенье ☒ понедельник ☐ вторник ☐ среда

☒ четверг ☐ пятница ☐ суббота

Дополнительные параметры

☐ Отложить задачу на (произвольная задержка): 1 ч.

☐ Повторять задачу каждые: 1 ч. в течение: 1 д.

☐ Останавливать все задачи по истечении срока повторов

☐ Остановить задачу через: 3 дн.

☐ Срок действия: 23.10.2026 13:47:58 ☐ Синхр. по поясам

☒ Включено

OK Отмена

Рисунок 1 – Параметры триггера

Теперь планировщик задач будет сохранять резервные копии БД еженедельно в понедельник и четверг, при этом каждый файл будет иметь свое уникальное имя.

Проверим, что бэкап-файлы верные и создаются верно. Для этого создадим новую БД с именем Test, выберем в СУБД функцию Restore, укажем файл .backup, созданный ранее, в качестве параметра для восстановления и подтвердим операцию нажатием клавиши

Restore. Далее, получаем успешные результаты (рисунок 2) и структуру со всеми пользователями, таблицами и триггерами, которые были в изначальной базе данных в новой БД Test (рисунок 3)

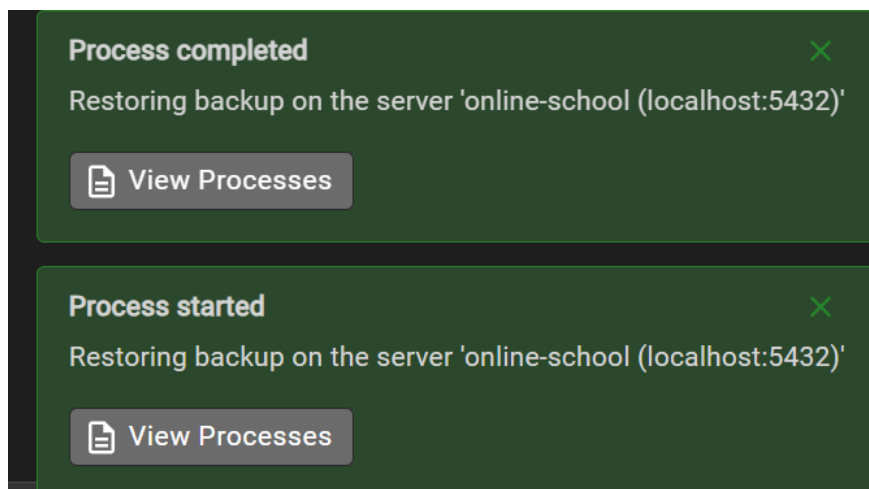


Рисунок 2 – Подтверждение операции

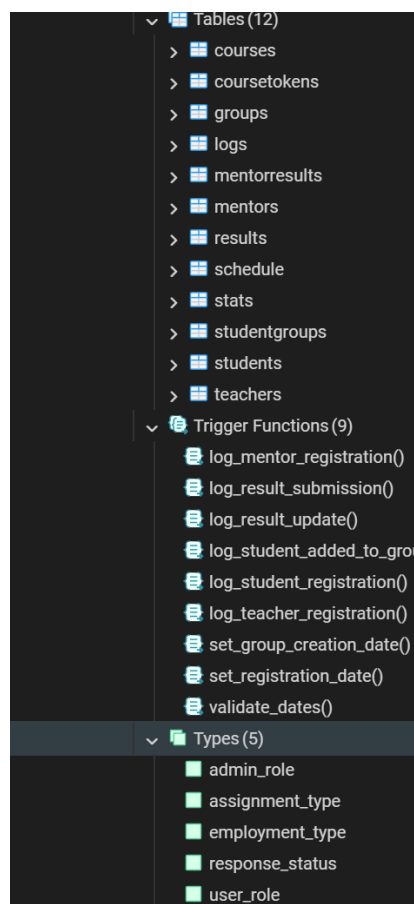


Рисунок 3 – Все отношения в новой БД Test

Таким образом, мы видим, что все .backup файлы создаются корректно и по ним можно восстановить исходную базу данных.

1.3 Внесение случайных изменений и откат к последней контрольной точке

Для выполнения данного задания внесем случайные изменения в одно из отношений исходной базы данных до создания контрольной точки. Допустим, внесем изменения в таблицу Students – заменим ФИО ученика на другое при помощи команды:

```
UPDATE students SET FIO = 'Иванов Олег Сергеевич' WHERE Studentid = 2;
```

Проверим:

```
select * from students where studentid = 2;
```

Получим вывод:

```
2      "Иванов Олег Сергеевич" "2007-09-12"      "zolinai@mail.ru"
      "+79601243275"          "2025-09-23"
```

Далее, создадим контрольную точку с правами суперпользователя:

```
onlineSchool=# CHECKPOINT;
CHECKPOINT
```

После этого проведем откат к предыдущей версии. Для этого удалим базу данных при помощи:

```
DROP DATABASE onlineSchool
```

После этого создадим БД с таким же именем, восстановим из .backup файла, созданного в предыдущем пункте, при помощи Restore и убедимся, что изменения не произошли:

```
select * from students where studentid = 2;
```

Получим вывод:

```
2      "Золин Алексей Игоревич" "2007-09-12"      "zolinai@mail.ru"
      "+79601243275"          "2025-09-23"
```

Таким образом, все изменения откатились к предыдущему состоянию.

1.4 Анализ внесенных изменений

1.4.1 Первый способ

Первый способ – проанализировать внесенные изменения при помощи журнала WAL, в который записываются все изменения, даже те, которые были позже отменены. Для этого можно воспользоваться утилитой pg_waldump при помощи команды, запускаемой в командной строке:

```
"C:\Program Files\PostgreSQL\17\bin\pg_waldump.exe" 00000001000000000000000004
```

Далее, можно увидеть последние успешные коммиты:

```
rmgr: Standby      len (rec/tot):    50/    50, tx:          0, lsn:
0/04E6B638, prev 0/04E6B618, desc: RUNNING_XACTS nextXid 8527
latestCompletedXid 8526 oldestRunningXid 8527
rmgr: XLOG         len (rec/tot):   114/   114, tx:          0, lsn:
0/04E6B670, prev 0/04E6B638, desc: CHECKPOINT_ONLINE redo 0/4E6B618; tli 1;
prev tli 1; fpw true; wal_level replica; xid 0:8527; oid 41106; multi 1;
offset 0; oldest xid 731 in DB 1; oldest multi 1 in DB 1; oldest/newest
commit timestamp xid: 0/0; oldest running xid 8527; online
rmgr: Standby      len (rec/tot):    50/    50, tx:          0, lsn:
0/04E6B6E8, prev 0/04E6B670, desc: RUNNING_XACTS nextXid 8527
latestCompletedXid 8526 oldestRunningXid 8527
rmgr: XLOG         len (rec/tot):   114/   114, tx:          0, lsn:
0/04E6B720, prev 0/04E6B6E8, desc: CHECKPOINT_SHUTDOWN redo 0/4E6B720; tli 1;
prev tli 1; fpw true; wal_level replica; xid 0:8527; oid 33935; multi 1;
offset 0; oldest xid 731 in DB 1; oldest multi 1 in DB 1; oldest/newest
commit timestamp xid: 0/0; oldest running xid 0; shutdown
pg_waldump: ошибка: ошибка в записи WAL в позиции 0/4E6B720: неверная длина
записи в позиции 0/4E6B798: ожидалось минимум 24, получено 0
```

(Последняя строка означает успешное завершение чтение журнала wal, “Invalid record length ... got 0” — indicates end of WAL file, not corruption. – из документации).

Далее, проанализируем журнал – в конкретно представленном выше отрезке нет успешных коммитов, только техническая информация, однако ранее по файлу мы видим, что раньше была создана контрольная точка, а до нее были внесены изменения.

1.4.2 Второй способ

Кроме журнала резервирования так же возможно посмотреть внесенные изменения при помощи таблицы logs, созданной в лабораторной работе №2. Для этого при создании .backup файла надо строго указать, какие таблицы должны откатываться, а какие должны сохранять свое состояние. В этом списке можно не выбирать таблицу logs, в результате чего далее мы сможем просматривать записи, которые при помощи триггера переходят в эту таблицу.

Так как мы уже вносили изменения в базу, а .backup файл был создан изначально с нужными параметрами, мы уже можем проверить, что было в таблице logs при помощи команды:

```
select * from logs
```

Получаем вывод:

```
60 13 Admin Изменение данных 2025-10-23 14:35:53.92672
```

Таким образом, просмотреть «откаченные» изменения можно без системы резервирования.

ЗАКЛЮЧЕНИЕ

В ходе работы была достигнута поставленная цель — приобретены практические навыки по резервированию и восстановлению базы данных в среде PostgreSQL. В процессе выполнения лабораторной работы были созданы и протестированы контрольные точки, продемонстрирована процедура отката к последней сохранённой версии базы, а также изучены механизмы анализа изменений в системе.

В результате выполнения задач были:

- изучены принципы работы контрольных точек (checkpoints) и их роль в обеспечении целостности данных;
- проверена возможность восстановления состояния БД из резервной копии, созданной при помощи утилиты `pg_dump`;
- исследованы возможности анализа изменений с использованием системных журналов WAL и пользовательской таблицы логирования, реализованной ранее.

Таким образом, в ходе работы было подтверждено, что PostgreSQL обеспечивает как физический уровень защиты данных через WAL-журнал и резервное копирование, так и логический уровень контроля изменений при помощи встроенных механизмов логирования. Полученные навыки позволяют не только выполнять восстановление данных после сбоев, но и анализировать историю операций, что является основой для построения отказоустойчивых и контролируемых информационных систем.